Commit Now, Walk Away Later: Designing Penalties for Delayed Blockchain Transactions

CS 6001 Course Project: Proposal

Reman Krishnan (Roll No. 26D0365)

Theme: Incentive Design in Blockchain Transaction Authorization (Self-Proposed)

1 Motivation and Problem Statement

Post-quantum signatures are larger in size compared to the digital signatures that are being used now in blockchains. A blockchain that switches to them has to move, check and store larger size of signature data for every transaction, and on a replicated ledger that cost would be paid by every node. One recent work addresses this problem by keeping the full post-quantum signature off the chain [1]. As per the proposed work, the user first posts a short commitment to the transaction. A few blocks later, the user reveals a short hash-based value that authorizes the committed transaction. Such designs are called as *deferred-authorization protocols*.

The delay saves space for a ledger to store signatures. However, it also creates an incentive problem.

While the transaction waits, the user can watch the chain. Suppose it is a trade and the price moves the wrong way. The user simply never authorizes it, and nothing happens. So the protocol has quietly given the user a *free option*, the right but not the obligation to go ahead after seeing how things turned out. This is not a new phenomenon. Cross-chain atomic swaps suffer from exactly the same thing, and Han, Lin and Yu estimated the option there at 2–3% of the swapped value for cryptocurrency pairs [2].

Some proposed designs try to close the gap with a *forfeit*, charged when a commitment is never authorized, and a *fallback path*, in which the block producer, who saw the user's full signature before accepting the commitment, publishes it and completes the transaction anyway. Both are mechanism design decisions. How big should the forfeit be? Who should get the money? Will the producer actually bother to publish? And what if the user simply pays the producer not to? The project studies these questions in this order.

Q1. Characterization (core). What is the smallest forfeit that makes committing just for the option unprofitable, and how does it depend on the window length k, price volatility and transaction size?

Q2. Mechanism design (core). Where should the forfeit go? Burning it, giving it to the producer, paying it into a common pool and returning it to the user are compared on whether both players' incentives survive.

Q3. Robustness (main extension). When does a producer holding the signature prefer to publish it, and when can the user and producer jointly profit by keeping the option alive?

Q4. Contestation (stretch goal). On ledgers without fast finality, is a bonded contest or a first-to-exhibit race the better way to settle a disputed commitment?

The model is small, but it has a sequential game, a random price and a designer who can only use penalties. The question of where the penalty goes also connects to budget balance and redistribution in VCG and Groves mechanisms.

2 Related Work

Base paper. The paper to be read in detail is by Xu, Ackerer and Dubovitskaya [3]. They treat a cross-chain atomic swap as a sequential game between two agents facing a random token price. At every stage each agent can continue or walk away, and backward induction gives the equilibrium. What they find is intuitive but important: once the price moves far enough, a rational agent abandons the swap, so success rates drop as volatility rises. Collateral helps. Delayed decisions and strategic abandonment are exactly the two ingredients of the present problem, which is why this paper is the starting point.

Four things change here. The game is lopsided, because only the user holds the option and the producer can take it away. That producer is a third party with its own incentives. The penalty has no natural owner, unlike swap collateral, which goes to the counterparty. And since the transaction is public at commit time, the forfeit can be tied to its size.

Pricing the option. Han, Lin and Yu [2] showed that an atomic swap is really an American call option sold without a premium, priced the missing premium with the Cox–Ross–Rubinstein binomial model [4], and built swaps that charge it. Xue and Herlihy [5] carried premiums over to multi-party deals. Dubovitskaya, Ackerer and Xu [6] found, for packetized payments, that collateral is again what keeps agents honest. From this line of work the project borrows a pricing method that can be checked, and the idea that a penalty can stand in for the premium.

Incentives of block producers. The producer side draws on transaction fee mechanism design. Roughgarden [7] introduced user incentive compatibility (UIC), miner incentive compatibility (MIC) and a collusion notion called OCA-proofness. Chung and Shi [8] asked for more, namely that no producer together with some users can gain by deviating as a group, and showed this is impossible under congestion. Chung, Roughgarden and Shi [9] later proved that even UIC, MIC and OCA-proofness cannot all hold when transactions compete for space. Q3 is a question of the same flavour, asked about a different mechanism.

Protocols and nearby problems. In the scheme of Finlow-Bates et al. [1], only the user can complete the reveal, so every transaction carries the option and nothing can force settlement. Daian et al. [10] give good reason not to treat producers as neutral. For the stretch goal, the nearest models are the verifier's dilemma [11], rollup dispute games [12] and watchtower incentives [13].

The gap. Swap papers price the option but have nobody who can cancel it. Fee mechanism papers model producers but not options created by delay. A preliminary search found nothing joining the two, and nothing that treats the routing of a forfeit as a design choice. Forward-citation searches from [3] and [2] in the first week will check this.

3 Proposed Approach

3.1 The model

A protocol is described by $(k, f, \rho, \mathcal{A})$, meaning the window length k in blocks, the forfeit rule f, the routing rule ρ and the set $\mathcal{A}$ of settlement paths. The game runs as follows.

1. At block 0 the user commits to a public transaction m and decides whether to hand the full signature σ to the producer.
2. Over blocks $1, \ldots, k-1$ the price S_t moves on a binomial lattice that both players can see. The transaction's value at execution is $g(m, S)$. As a modelling assumption, $\mathbb{E}[g] = 0$ for a trade, and g is constant for a plain transfer.
3. The user may reveal at any block up to $k-1$. Each attempt gets in with probability $1-q$, so waiting for information carries a risk of missing the deadline.
4. At block k, if nothing has been revealed and the producer holds σ, it decides whether to publish. Publishing earns the fee φ and costs c in block space.
5. Otherwise the transaction is dropped and the account pays $f(m)$. The rule ρ sends this money to one of four places: back to the user, burned, to the producer or to a pool.

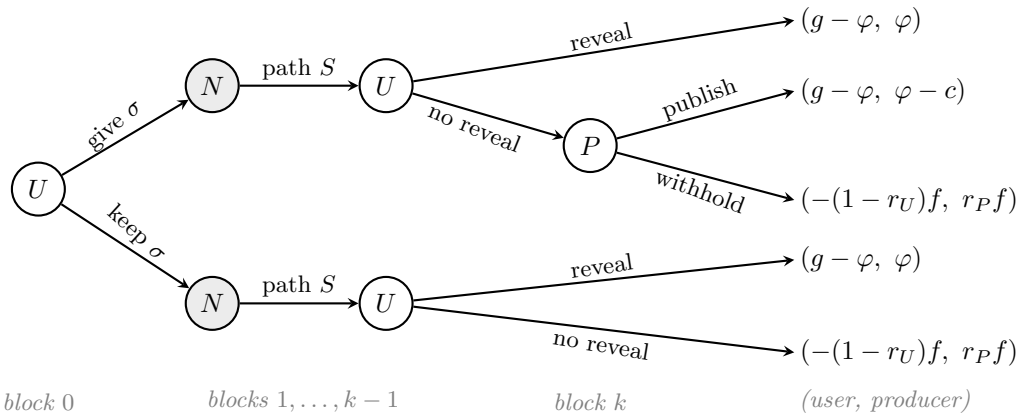


Figure 1: The base game, simplified. U is the user, P the producer and N the chance move drawing the price path S. In the full model the user decides block by block and each reveal lands with probability $1-q$. Here $r_U = 1$ if ρ returns the forfeit to the user, $r_P = 1$ if ρ pays the producer, and both are zero otherwise.

Figure 1 shows the game in simplified form. Utilities are quasi-linear. Both players see the same price path, so the base model is an extensive-form game of complete information with chance moves, and the solution concept is subgame-perfect equilibrium. Q3 adds joint deviations with side payments. A Bayesian version, where the user has a private type $\theta_u \sim D$ such as a hidden exposure, will be added if time permits.

3.2 Starting observations

Three observations follow directly from the model. They are not results yet. Each still needs a proof and a computational check.

Observation 1 (Self-routing has no effect)**.** *If the forfeit goes back to the account whose commitment went unauthorized, and only that account holder can authorize it, the user is paying the user. Nothing changes.*

Observation 2 (The speculator's condition)**.** *Take a speculator with no real reason to trade, who commits and authorizes only when executing beats paying the forfeit. Since* $\max(a, b) = b + (a-b)^+$*, the expected profit is*

$$\Pi(f) = \mathbb{E}\big[\max(g - \varphi, -f)\big] = -f + \mathbb{E}\big[(g - (\varphi - f))^+\big].$$

The second term is the value of a call on g *with strike* $\varphi - f$*, so speculation stops paying exactly when that call is worth at most* f*. With* $\mathbb{E}[g] = 0$*, put-call parity gives the equivalent form* $\Pi(f) = \mathbb{E}[((\varphi - f) - g)^+] - \varphi$*.*

Two things follow. A flat forfeit cannot stop large trades, because g grows with transaction size. Since m is public, though, the forfeit can grow with it. The option is also worth more when the window is longer, so the required forfeit rises with k.

Observation 3 (Producer routing can disable the fallback)**.** *Suppose the producer holding* σ *collects the forfeit when the transaction fails, all else fixed. Publishing gives it* $\varphi - c$ *and withholding gives it* f*. If* $f > \varphi - c$*, it never publishes.*

So the forfeit is squeezed from both sides. It has to be large enough for Observation 2, small enough that an honest user whose reveal gets censored is not badly hurt, and sent somewhere that does not trigger Observation 3. Whether anything fits, and for which parameters, is what Q1 and Q2 are really about.

3.3 Planned results

Q1 and Q2 are the core and will be finished first. Q3 comes next. Q4 happens only if there is time.

- **Q1.** For a fixed routing rule ρ, what is the smallest forfeit that makes committing just for the option unprofitable, and how does it depend on the window length k, price volatility and transaction size?
- **Q2.** Compare the four routing rules, with self-routing as the baseline. A good rule deters speculation, keeps the producer willing to publish and limits what an honest user can lose. The working hypothesis is that burning does all three.
- **Q3.** Find when the user and producer cannot jointly gain from a side payment that keeps σ unpublished, by testing the conjecture below.

Conjecture 1. *Under producer routing, no forfeit rule can deter the user, keep the producer willing to publish and resist user-producer side contracts all at once. Under burn routing, all three hold over a non-empty range of parameters, and that range shrinks as volatility and* k *grow.*

The conjecture may be false. In that case, a counterexample will be treated as the result. In that case the counterexample is the result. Likewise, if the Bayesian version has no clean closed form, it will be reported numerically.

3.4 Implementation and verification

Everything below is planned to be implemented in Python, with NumPy and SciPy.

- **Deviation checker.** To be written first. On small lattices with discretized forfeits and side payments, it tries every unilateral deviation and every joint user-producer deviation and flags any that pay. It should catch plain commit-reveal, with no forfeit and no fallback, as exploitable, and that is its first test.
- **Game simulator.** Optimal reveal policy, producer best response and subgame-perfect equilibrium outcomes for any $(k, f, \rho, \mathcal{A})$. Ties are recorded, not broken silently, so multiple equilibria stay visible.
- **Checks against known answers.** The lattice against the closed-form call case. The pricing code against the premium estimates of Han et al. [2].
- **Real data.** Volatility from public historical prices at block-scale intervals, such as twelve seconds, then a map of where a feasible forfeit exists across k and transaction size.
- **Reproducibility.** A public repository with one script that regenerates every figure and table.

3.5 Relevance to course material

The core of the project is Part II of the course: extensive-form games, subgame perfection and backward induction, plus Bayesian games for the optional extension. Part III gives the mechanism design view, including incentive compatibility and individual rationality for honest users. From Part IV come quasi-linear utilities and payments. Option pricing and market data are outside the syllabus, but they are only used to calibrate and check the results.

4 Expected Timeline

Dates	Milestone
Sep 28 – Oct 4	*Sketch and survey.* Detailed reading of [3]. Forward-citation search from [3] and [2], narrowed to the two or three most relevant papers. Model fixed. Deviation checker written and tested.
Oct 5 – Oct 11	*Build and verify.* Simulator, reveal solver and producer best response. Benchmark checks passing. Proofs of Observations 1 to 3.
Oct 12 – Oct 18	*Core: Q1 and Q2.* Characterization of f^* and the feasible range. Comparison of routing rules. Calibration on real price data.
Oct 19 – Oct 24	*Extension: Q3.* Side-contract analysis and testing of the conjecture. Q4 only if all of the above is complete.
Oct 25 – Oct 27	*Write-up.* Report within the 10-page limit, with the repository and reproduction script.
Oct 28 – Oct 30	Presentation preparation.

References

[1] T. Finlow-Bates, M. Jakobsson and H. Siadati, *The cost of quantum resistance: a hash-based commit-reveal alternative for minimizing blockchain infrastructure overhead*, arXiv:2605.06853, 2026.

[2] R. Han, H. Lin and J. Yu, *On the optionality and fairness of atomic swaps*, ACM Conference on Advances in Financial Technologies (AFT), 2019, pp. 62–75.

[3] J. Xu, D. Ackerer and A. Dubovitskaya, *A game-theoretic analysis of cross-chain atomic swaps with HTLCs*, IEEE International Conference on Distributed Computing Systems (ICDCS), 2021, pp. 584–594.

[4] J. C. Cox, S. A. Ross and M. Rubinstein, *Option pricing: a simplified approach*, Journal of Financial Economics, 7(3):229–263, 1979.

[5] Y. Xue and M. Herlihy, *Hedging against sore loser attacks in cross-chain transactions*, ACM Symposium on Principles of Distributed Computing (PODC), 2021, pp. 155–164.

[6] A. Dubovitskaya, D. Ackerer and J. Xu, *A game-theoretic analysis of cross-ledger swaps with packetized payments*, arXiv:2103.02056, 2021.

[7] T. Roughgarden, *Transaction fee mechanism design*, ACM Conference on Economics and Computation (EC), 2021.

[8] H. Chung and E. Shi, *Foundations of transaction fee mechanism design*, ACM-SIAM Symposium on Discrete Algorithms (SODA), 2023, pp. 3856–3899.

[9] H. Chung, T. Roughgarden and E. Shi, *Collusion-resilience in transaction fee mechanism design*, ACM Conference on Economics and Computation (EC), 2024, pp. 1045–1073.

[10] P. Daian, S. Goldfeder, T. Kell, Y. Li, X. Zhao, I. Bentov, L. Breidenbach and A. Juels, *Flash Boys 2.0: frontrunning in decentralized exchanges, miner extractable value, and consensus instability*, IEEE Symposium on Security and Privacy (S&P), 2020, pp. 910–927.

[11] L. Luu, J. Teutsch, R. Kulkarni and P. Saxena, *Demystifying incentives in the consensus computer*, ACM Conference on Computer and Communications Security (CCS), 2015, pp. 706–719.

[12] H. Kalodner, S. Goldfeder, X. Chen, S. M. Weinberg and E. W. Felten, *Arbitrum: scalable, private smart contracts*, USENIX Security Symposium, 2018, pp. 1353–1370.

[13] P. McCorry, S. Bakshi, I. Bentov, S. Meiklejohn and A. Miller, *Pisa: arbitration outsourcing for state channels*, ACM Conference on Advances in Financial Technologies (AFT), 2019, pp. 16–30.